Data Structure

2026-1학기 부경대학교 알고리즘 스터디

목차

- 자료구조의 정의
- 선형 자료구조
  - Array
  - Linked List
  - Stack
  - Queue
- 비선형 자료구조
  - 트리
  - 그래프
- 추상적 자료형
  - 정의
  - Set
  - Map
- 구현체
  - Hash
  - BST

정의

- 자료구조(data structure)란?
  - 데이터를 디스크(메모리)에 저장하고 조작하는 구조
  - 적절한 자료구로를 사용해 삽입, 삭제, 갱신, 탐색 등의 연산을 효율적으로 수행
- 자료구조를 배우는 이유?
  - 자료구조마다 제공하는 연산이 다르고, 같은 연산이더라도 시간 복잡도가 다름
  - 문제가 어떤 연산(삽입, 삭제, 대입, 질의 등)을 요구하는지에 따라 효과적인 자료구조가 존재하고, 적절한 자료구조를 사용해 시간 복잡도를 줄일 수 있음
  - **즉 자료구조를 공부한다는 것은 문제를 코드가 아니라 연산과 병목으로 바라보는 사고를 기르는 것!**

선형 자료구조

- 선형(**線型**)성 : 직선처럼 올곧은 성질을 가진 것
  - 수학적으로는 직선인 모양 (혹은 도형)을 의미함
- 선형 (linear) 자료구조란?
  - 데이터를 한 줄로 나열하여 저장하는 구조
  - 자료들의 앞, 뒤 관계가 1 : 1 구조
  - Array, Stack, Queue 등이 선형 자료구조에 속함
  - 데이터간 단일한 순서성이 존재

선형 자료구조 - Array

- Array(배열)이란?
  - **같은 타입**의 데이터를 **연속된 메모리 공간**에 나란히 저장한 자료구조
  - 각 원소를 **인덱스**로 접근 가능

    ※ C, C++에서는 첫번째 원소의 인덱스를 0으로 설정 (0-based Index)
  - 고정된 크기

    ※ C++ STL에는 vector라는 동적 배열 타입이 존재
  - 논리적 저장 순서와 물리적 저장 순서가 동일

Array

5	8	3	12	7
0	1	2	3	4

선형 자료구조 - Array

- Array 연산의 시간 복잡도
  - 길이가 N인 배열의 i번째 인덱스에 다음과 같은 연산을 수행한다고 가정해보자.
  - 접근 : O(1)
  - 갱신 : O(1)
  - 삽입 및 삭제 : O(N)

    ※ 데이터를 앞, 중간 삽입할 경우 그 뒤의 원소들을 한칸씩 밀어야 하므로 O(N)의 시간 복잡도를 가짐
    ※ 마찬가지로 데이터를 앞, 중간 삭제할 경우 그 뒤의 원소들을 한칸씩 당겨야 하므로 O(N)의 시간 복잡도를 가짐
    ※ 크기가 고정적이므로 자료구조 차원에서 완전한 삽입 및 삭제 연산을 지원하지 않음

선형 자료구조 – Array

- 2차원 배열
  - 배열의 인덱스가 두 정수로 표현되는 배열
  - int arr[a][b]의 경우 길이가 b인 배열이 총 a개 모인 것(a * b)으로 볼 수 있음
  - 격자 모양의 배열로 a[i][j]와 같은 방법으로 접근 가능
  - 1차원 배열과 마찬가지로 메모리에 연속적으로 저장됨

    ※ 언어 방식에 따라 행 우선인 경우 존재
  - 경우에 따라서 3차원 이상의 고차원 배열도 선언 가능

선형 자료구조 - Array

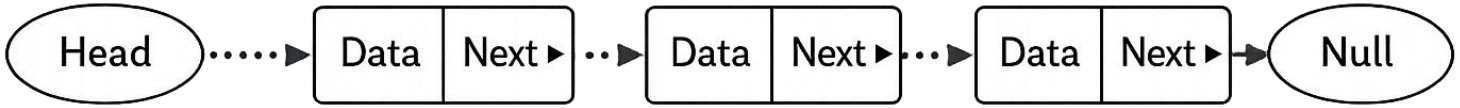
- **C++에서의 배열**
  - **배열이름[인덱스]** 로 원소에 접근 가능
  - 인덱스가 **0부터** 시작
  - **배열의 이름**이 대부분 배열의 **첫번째 원소의 포인터**를 반환
  - 가령 &arr[3]의 경우 (arr + 3)와 동일
  - 2차원 배열에서 &arr[3][2]의 경우 *(arr + 3) + 2와 동일

```cpp
int main() {
    // 크기가 5인 int형 배열 선언
    int A[5] = {1, 3, 5, 7, 9};


    // A[2]에 11 대입
    A[2] = 11;

    for(int i = 0 ; i < 5; i++)
        cout << A[i] << " ";

    cout << endl;

    // 크기가 3 x 3인 2차원 배열 선언
    int B[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // 2차원 배열 접근후 값 대입
    B[2][1] = 0;

    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            cout << B[i][j] << " ";
        }
        cout << endl;
    }

}
```

```
1 3 11 7 9
1 2 3
4 5 6
7 0 9
```

선형 자료구조 – Linked List

- Linked List(연결 리스트)란?
  - **(값, 다음 노드의 주소)**를 하나의 노드로 저장해 순차를 유지하는 자료구조
  - 자료구조 차원에서 **삽입/삭제 연산** 제공
  - 논리적 선형성을 가지나, 데이터들의 **물리적인 위치는 연속적이지 않음**
  - 이전 원소의 주소를 포함하여 Double Linked List 자료구조도 표현 가능
  - Linked List의 시작 노드를 **Head**, 마지막 노드를 **Tail**이라고 함



선형 자료구조 – Linked List

- Linked List 연산의 시간 복잡도
  - 길이가 N인 배열의 i번째 인덱스에 다음과 같은 연산을 수행한다고 가정해보자
  - 접근 : O(N)
    ※ i번째 원소까지 순회해야하므로 O(N)
  - 갱신 : O(N)
    ※ i번째 원소에 접근 (O(N)), 값 갱신 O(1)
  - 삽입 : O(N)
    ※ i번째 원소에 접근 (O(N)), 인덱싱 O(1)
  - 삭제 : O(N)
    ※ i번째 원소에 접근 (O(N)), 인덱싱 O(1)
  - 추가 : O(1)
    ※ Tail 노드의 주소를 알 경우 O(1), 그렇지 않을 경우 삽입 연산과 동일

선형 자료구조 – Linked List

- C++ 에서의 Linked List
  - C++ STL의 std :: forward_list가 linked list로 구현되어 있음
  - 각 원소에 반복자 (iterator)로 접근

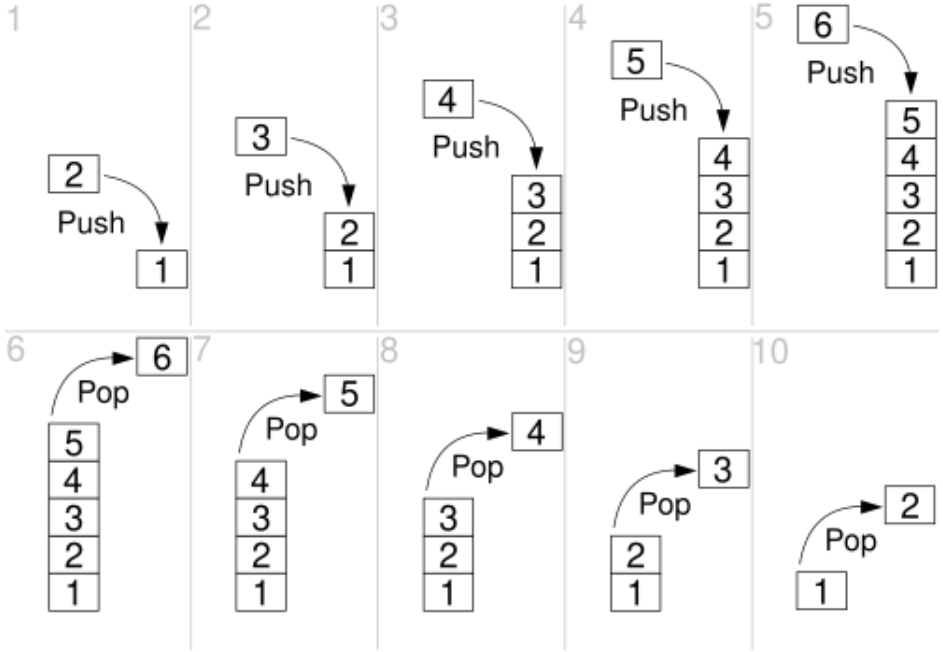
    ※ 반복자에 대한 설명은 부록 Cpp for PS 참조

```cpp
int main() {
    forward_list<int> linked_list;
    // 삽입
    // 1. 맨 앞 삽입
    linked_list.push_front(3);
    linked_list.push_front(5);
    linked_list.push_front(7);

    // 2. 중간 삽입
    // (반복자, 삽입할 값)을 인자로 받아 해당 반복자 다음에 삽입
    linked_list.insert_after(linked_list.begin(), 23);

    cout << "삽입 후 결과 : " << endl;
    for(auto i = linked_list.begin(); i != linked_list.end(); ++i)
        cout << *i << " ";
    cout << endl;

    // 삭제
    // 1. 맨 앞 삭제
    linked_list.pop_front();

    // 2. 반복자 값을 인자로 받아 해당 반복자 다음에 삭제
    linked_list.erase_after(linked_list.begin());

    cout << "삭제 후 결과 : " << endl;
    for(auto i = linked_list.begin(); i != linked_list.end(); ++i)
        cout << *i << " ";

    cout << endl;
}
```

```
삽입 후 결과 :
7 23 5 3
삭제 후 결과 :
23 3
```

선형 자료구조 - Stack

- Stack이란?
  - **LIFO** (Last-In, First-Out) 규칙을 가진 자료구조
  - Array 혹은 Linked List로 쉽게 구현이 가능



- Stack 연산
  - push() : 스택에 값을 추가
  - pop() : 가장 최근에 추가된 값을 제거
  - peak() : 가장 최근에 추가된 값을 반환

선형 자료구조 - Stack

- PS에서의 Stack 응용
  - 간단한 구조 덕분에 다양하게 활용됨
  - 괄호 검증

    ※ (), {} 와 같은 괄호식이 올바른지 스택으로 검증
  - 수식 표현식

    ※ 연산자 우선순위를 스택을 통해 표현 후 처리할 수 있음
  - 깊이 우선 탐색

    ※ 재귀 함수 대신 스택을 사용해 그래프에서 우선 순위 탐색 표현 가능
  - 그 외에도 프로세스의 콜 스택, 프로그래밍 언어의 구문 분석 등 다양한 분야에서 사용되는 자료구조!

선형 자료구조 - Stack

- C++ 에서의 Stack
  - 템플릿 원형 : std::stack<T, Container>

    ※ T는 스택에 사용할 데이터 타입, 컨테이너는 내부 구현체를 의미함

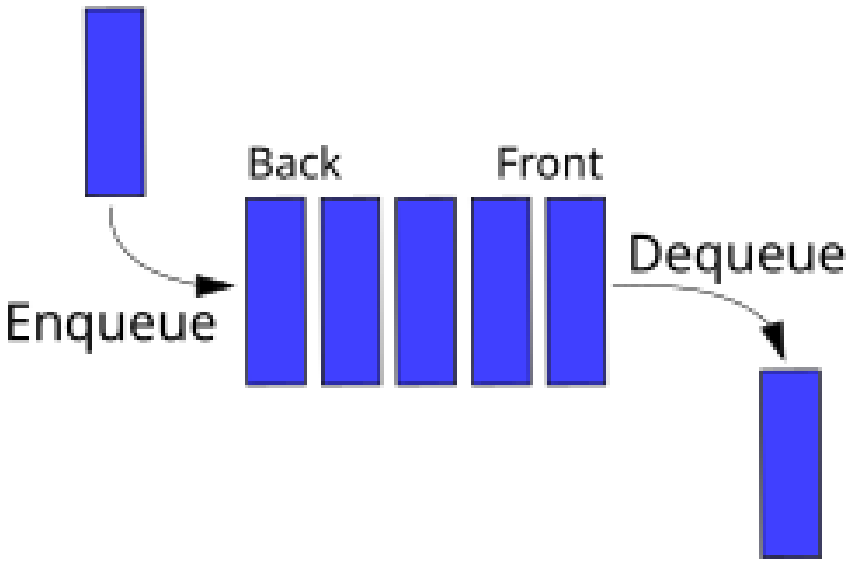
  - pop() : stack의 pop 연산을 제공
  - push() : stack의 push 연산을 제공
  - top() : stack의 peak 연산을 제공
  - size() : 원소의 개수 리턴
  - empty() : stack의 원소의 개수가 0개면 true, 1개 이상이면 false를 리턴

```
int main() {
    stack<int> st;

    // 스택에 순서대로 1, 3, 5 추가
    st.push(1);
    st.push(3);
    st.push(5);

    // 출력
    while(!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
}
```

```
5 3 1 %
```

선형 자료구조 - Queue

- Queue란?
  - **FIFO** (First-In, First-Out) 규칙을 가진 자료구조
  - 데이터가 들어가는 위치를 Back 혹은 Rear로 표현
  - 데이터가 나가는 위치를 Front로 표현
- Queue 연산
  - enqueue() : 큐에 데이터를 삽입
  - dequeue() : 가장 마지막에 추가된 값 제거



선형 자료구조 - Queue

- 다양한 형태의 Queue
  - 환형 큐 (Circular Queue)
    - ※ 기존의 Queue를 배열로 구현할 경우 데이터가 밀리고, 원소의 개수보다 더 큰 용량의 배열을 선언해야 하는 문제가 발생함
    - ※ 이를 해결하기 위해 rear와 front의 포인터를 모듈러 연산을 통해 회전시켜 크기가 고정적인 배열로 큐를 표현
  - 데크 (Deque)
    - ※ Queue와 달리 rear, front에서 모두 삽입, 삭제가 가능한 자료구조

선형 자료구조 - Queue

- C++ 에서의 Queue
  - 템플릿 원형 : std::queue<T, Container>
  - pop() : queue의 dequeue 연산을 제공
  - push() : queue의 enqueue 연산을 제공
  - back() : queue의 맨 뒤 원소 참조
  - front() : queue의 맨 앞 원소 참조
  - stack 과 동일하게 empty(), size() 함수 제공

```
int main() {
    queue<int> q;

    // 삽입
    q.push(1);
    q.push(2);
    q.push(3);

    // 큐가 빌 때까지 dequeue
    while(!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }
}
```

```
1 2 3 %
```

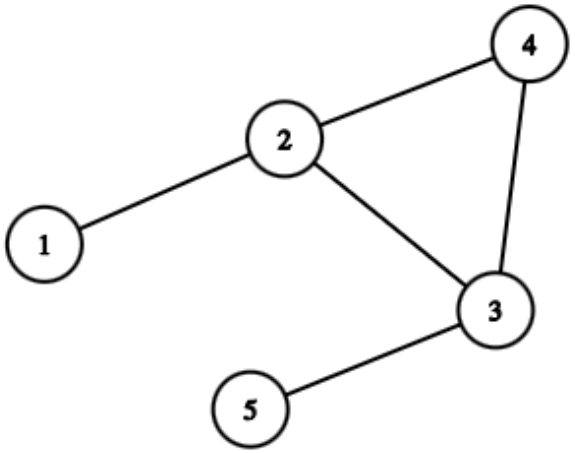
비선형 자료구조

- 하나의 원소가 여러 개의 원소와 연결될 수 있는 자료구조
- 계층 (트리), 망 (그래프) 형태의 관계를 표현하는 자료구조

비선형 자료구조 - Graph

- Graph(그래프)란?
  - **정점**(Vertex)와 정점을 잇는 **간선**(Edge)로 구성된 자료구조
  - 도시-도시, 사람-사람 등 다양한 현상을 표현할 수 있는 자료구조
  - 방향 그래프 : 간선에 방향이 있는 그래프
  - 무방향 그래프 : 간선에 방향이 없는 그래프
  - 가중치 그래프 : 간선에 가중치가 존재하는 그래프

    ※ 가령, 도시(정점)와 도시를 잇는 도로(간선)가 길 수록 가중치를 높게 부여해서 그래프로 표현할 수 있음



비선형 자료구조 - Graph

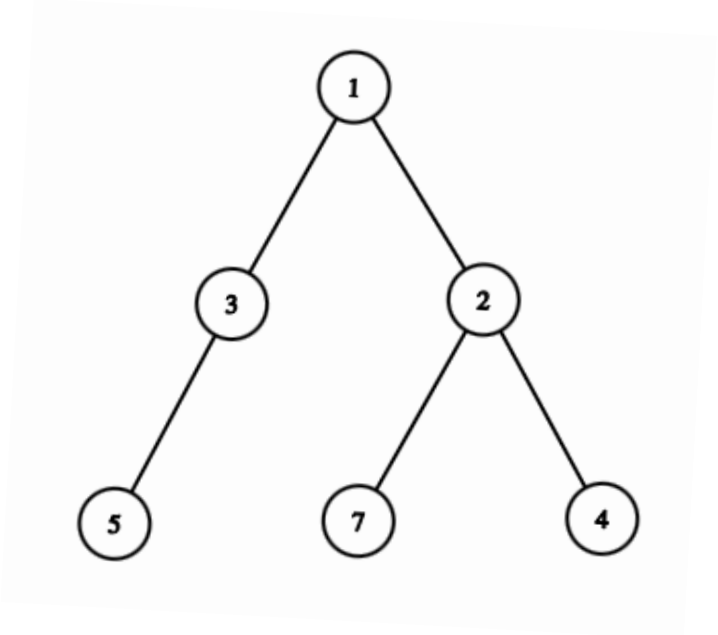
- Graph 용어
  - 정점 (Vertex) : 노드(Node)라고도 불리며 하나의 개체를 표현한 점
  - 간선 (Edge) : 정점과 정점 사이를 이어주는 선
  - 차수 (Degree) : 하나의 정점에 인접한 정점의 수
  - 경로 (Path) : 한 정점에서 출발하여 간선을 통해 다른 정점으로 가는 순서의 나열

    ※ 중간에 같은 정점을 두 번 이상 방문하지 않는 경로를 단순 경로라고 함
  - 순환 (Cycle) : 시작 정점과 끝 정점이 같은 단순 경로

비선형 자료구조 - Tree

- Tree(트리)란?
  - 다음 조건을 만족하는 그래프를 트리라고 정의
  - 1) 연결 그래프이다

    ※ 임의의 서로 다른 두 정점을 이어주는 경로가 존재한다.
  - 2) 사이클이 없다 (acyclic)
  - 디렉토리 구조, 이진 탐색 트리, 조직도 등 계층 구조 혹은 의사 결정

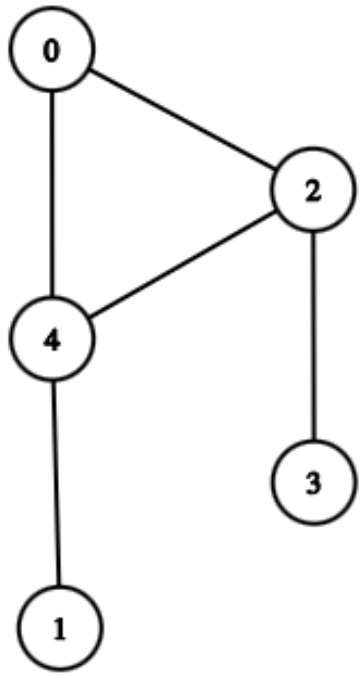


비선형 자료구조 - Tree

- Tree 용어
  - 루트 (root) : Tree에서 기준으로 잡은 정점
  - 부모 (Parent) : 루트에서 정점 x로 가는 경로에서 x 바로 직전의 정점
  - 자식 (Child) : 정점 x의 바로 아래에 연결된 정점
  - 리프 (leaf) : 자식이 없는 정점
  - 서브 트리 (Subtree) : 임의의 정점 x를 루트로 했을 때 x와 그 아래의 정점 및 간선으로만 이루어진 트리
  - 깊이 (depth) : 루트에서 정점 x까지의 거리
  - 높이 (height) : 깊이가 가장 큰 노드까지의 깊이

비선형 자료구조

- C++ 에서의 그래프, 트리
  - 직접 해당 자료구조를 표현하는 STL 컨테이너는 없음
  - **인접 행렬**, 혹은 **인접 리스트**로 구현



- 인접 행렬 vs 인접 리스트
  - 옆의 무방향 그래프를 각각 인접 행렬과 인접 리스트로 표현한다고 가정

비선형 자료구조

- 인접 행렬
  - 두 정점 a, b를 이어주는 간선이 있다면 adj_matrix[a][b] = 1로 설정
  - 장점
  - 1) 두 정점 사이의 간선이 존재하는지 확인하기 쉬움 (O(1))
  - 2) 구현이 간단

```
// 인접 행렬
int adj_matrix[5][5] = {
    {0, 0, 1, 0, 1},
    {0, 0, 0, 0, 1},
    {1, 0, 1, 0, 1},
    {0, 0, 1, 0, 0},
    {1, 1, 1, 0, 0}
};
```

  - 단점
  - 1) 공간 복잡도가 O(N^2)-> 단순 그래프일 수록 메모리 낭비가 커짐

비선형 자료구조

- 인접 리스트
  - 한 정점 a와 인접한 모든 노드의 번호를 vector (혹은 리스트)로 표현
  - 장점
  - 1) 간선의 수만큼 공간 사용하므로 효율적 (O(N + M))
  - 2) 인접 정점 순회가 빠름
  - 단점
  - 1) 두 정점 사이의 간선 존재 유무 확인이 어려움 (O(N))

```
vector<int> adj_list[5];
adj_list[0].push_back(2);
adj_list[0].push_back(4);

adj_list[1].push_back(4);

adj_list[2].push_back(0);
adj_list[2].push_back(3);
adj_list[2].push_back(4);

adj_list[3].push_back(2);

adj_list[4].push_back(0);
adj_list[4].push_back(1);
adj_list[4].push_back(2);
```

추상적 자료형 – 정의

- 추상적 자료형(ADT, Abstract Data Type)란?
  - 자료구조의 세부 구현은 은닉하고, 자료형이 제공하는 연산이 어떠한 기능을 하는지 수학적으로 정의한 자료형
  - 예) 스택의 경우 내부 구현에 상관없이 pop, push, top만 제공하면 되는 ADT
  - 가령 STL에서 제공하는 stack 컨테이너의 경우, 내부 구현의 디폴트는 deque

    ※ std::stack<T, deque<T>> 가 기본 템플릿 인자이므로, vector<T> 혹은 list<T>로도 구현 가능

- 연관 배열(Associative Array)이란?
  - 하나의 키 값에 연관되는 값들을 1 : N 구조로 저장하거나, 키 값을 조회할 수 있는 자료구조
  - C++에서 Associative Container에서 다양한 연관 배열 기능들을 제공
  - Associative Container는 자료를 정렬된 상태를 유지하며, BST(Balanced Binary Search Tree) 구조로 구성됨

추상적 자료형 - set

- set이란?
  - set은 Associative Containers에서 제공하는 추상적 자료형
  - key를 중복되지 않도록 저장하는 자료형
  - 주로 값을 저장한 다음 값의 존재 유무를 검색하기 위해 사용
  - 삽입, 삭제, 검색 세가지 연산을 지원함 (세 연산 모두 O(log N))

    ※ 세 연산의 시간 복잡도는 구현체 파트에서 다룰 예정

추상적 자료형 – set

- set의 메소드
  - insert(key) : key를 set에 삽입
  - find(key) : key가 set에 존재하는지 검색. 만약 존재하지 않는다면 end() iterator 반환
  - erase(key) : set에서 key를 삭제. 인자로 iterator를 받을 수 있음
  - end() : set의 마지막 원소 다음 iterator를 반환
  - begin() : set의 시작 iterator를 반환

```cpp
#include <set>

int main() {
    set<int> a;

    // a에 순서대로 3, 1, 5, 5삽입
    a.insert(3);
    a.insert(1);
    a.insert(5);
    a.insert(5);

    // 중복값은 한번만 삽입되고, 키값이 내부에서 정렬되므로 1 3 5가 출력됨
    // 마지막 iterator 이후의 iterator가 a.end()이므로, 반복문 종료 조건으로 사용됨
    for(auto i = a.begin(); i != a.end(); ++i) {
        cout << *i << " ";
    }
    cout << endl;

    // erase에 iterator를 인자로 넘겨줌
    a.erase(a.begin());
    // erase에 키값을 인자로 넘겨줌
    a.erase(3);


    // a의 시작 iterator의 값 1과 3을 삭제했으므로 5만 출력됨
    for(auto i = a.begin(); i != a.end(); ++i) {
        cout << *i << " " ;
    }

    return 0;
}
```

```
1 3 5
5 %
```

추상적 자료형 - map

- map이란?
  - Map은 Associative Containers에서 제공하는 추상적 자료형
  - key와 key에 대응되는 데이터를 **(key, value)** 형태로 저장한 자료구조
  - set과 마찬가지로 검색과 조회를 빠르게 하기 위해 사용됨
  - (학번, 이름) 과 같이 키 값에 대응되는 값의 조회가 자주 일어나는 상황에서 사용하기 좋은 자료구조
  - set과 마찬가지로 insert, erase, find 세 가지 기본 연산을 지원
  - Python의 dict와 유사한 기능들을 수행

추상적 자료형 – map

- Map의 메소드
  - insert({key, value}) : key와 key에 대응되는 value를 map에 입력
  - find(key) : key에 대응되는 value 검색. 만약 key가 없다면 end() iterator 반환
  - erase(key) : key에 대응되는 (key, value) 쌍을 제거. key 대신 iterator를 인자로 받을 수 있음
  - end() : 마지막 원소 다음의 iterator 반환
  - begin() : 시작 원소의 iterator 반환

```cpp
#include <map>

int main() {

    // key, value의 데이터 타입을 제너릭으로 받아서 정의 가능
    map<string, int> mp;

    // mp에 pair로 (key, value)를 입력
    mp.insert(make_pair("하승민", 94));
    mp.insert(make_pair("이명준", 96));
    mp.insert(make_pair("이지민", 99));
    mp.insert(make_pair("이정주", 93));

    // [] 연산자로 key에 대응되는 value에 대입 연산
      mp["이명준"] = 85;

    // key값을 인자로 전달해 ("하승민", 94) 데이터 삭제
    mp.erase("하승민");

    // 내부는 key값 기준으로 오름차순 정렬되므로, 사전순으로 출력
    for(auto i = mp.begin(); i != mp.end(); ++i) {
        cout << i->first << " " << i -> second << "\n";
    }
    return 0;
}
```

```
이명준 85
이정주 93
이지민 99
```

구현체

- 구현체란?
  - 구현체는 추상적 자료형이 제공하는 연산을 실제로 구현한 형태
  - 가령 Stack의 push(), pop(), peak() 연산을 Array로 구현했다면 Array가 구현체

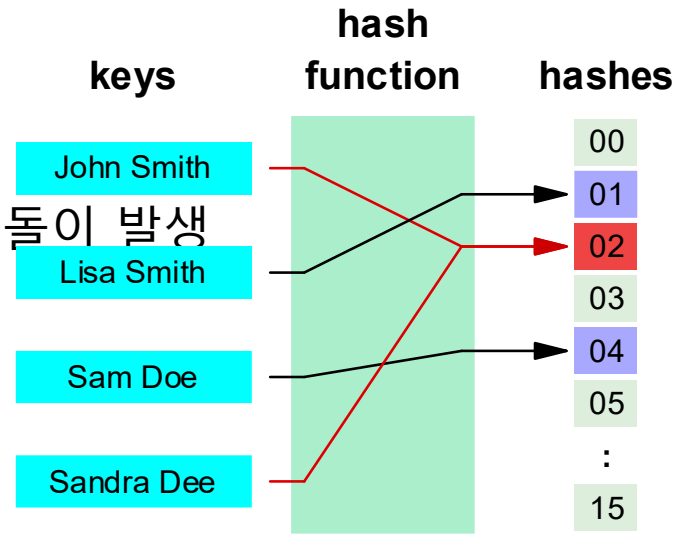
구현체 - hash

- hash란?
  - hash는 key를 해시 함수를 통해 인덱스를 생성해 저장 위치를 결정하는 방식
  - 해시 함수란 입력 공간 U에 대해 어떤 출력 공간 (정수 인덱스 혹은 고정 길이의 해시값) R로 매핑하는 함수

    ※ h : U->R
  - 주로 곱셈, 덧셈, xor 등의 연산을 여러번 거친 뒤 모듈러 연산을 통해서 정수 인덱스를 생성
  - **삽입, 삭제, 조회 연산을 다른 자료구조보다 효과적으로 수행**

구현체 - hash

- 해시 함수
  - key값을 연산을 통해 고정 길이의 해시 값 혹은 인덱스를 생성
  - key의 길이 수와 무관하게 고정된 길이를 리턴 + 비선형 연산때문에 충돌이 발생



- 해시 테이블
  - key를 해싱해 정수(인덱스)로 만들고, 해당 인덱스에 해당하는 버킷에 매핑하는 자료구조

    ※ 버킷은 해시 테이블의 한 칸(인덱스)로, 충돌한 데이터들의 저장 단위

  - 체이닝, 오픈 어드레싱 등 충돌을 처리하는 규칙과 함께 정의 됨

    ※ 체이닝은 linked list로 충돌 된 데이터들을 관리

구현체 – hash

- hash를 쓰는 이유?
  - 가령 크기가 N이고 정렬되지 않은 배열에서 key값이 존재하는지 조회한다고 가정해보자
  - 최악의 경우 모든 원소를 찾아야 함 -> O(N)

  - N개의 동일한 데이터를 해시 테이블에 저장한 상태에서, key값이 존재하는지 조회한다고 가정해보자
  - hash 테이블의 경우 key를 해싱하여 얻은 인덱스로 이동함
  - 버킷의 개수가 M이라고 하면, 각 버킷당 평균 부하율 a는 약 N / M
  - 즉 N개의 데이터가 아니라 N / M개의 데이터를 조회하면 됨

    ※ 체이닝 방법으로 충돌 데이터들을 관리할 경우
  - 내부적으로 a가 상수 수준의 크기가 되도록 M을 조절하므로 O(1 + a) = O(1)

구현체 - hash

- hash를 쓰는 이유?
  - 데이터를 삽입한다고 가정해보자.
  - key 해싱 비용 + 버킷 삽입 비용 (O(1)) 으로 효과적
    - ※ 해싱 알고리즘 자체는 key의 모든 비트를 한번씩 읽어야 하므로 하한이 O(비트 길이 수)
    - ※ PS에서는 key가 주로 int형 (32비트) 이므로 상수로 취급해도 무관함
  - 그러나 데이터를 해시 테이블에 삽입할 경우 버킷이 가득 찰 수 있음-> M의 크기를 조절 (O(N))
  - 버킷의 크기 조절은 가끔 발생하므로, 삽입의 경우 amortized(O(1))
    - ※ Amortized는 알고리즘을 여러번 수행할 때의 평균 시간 복잡도를 의미
  - 마찬가지로 데이터의 삭제 또한 amortized(O(1))

구현체 - hash

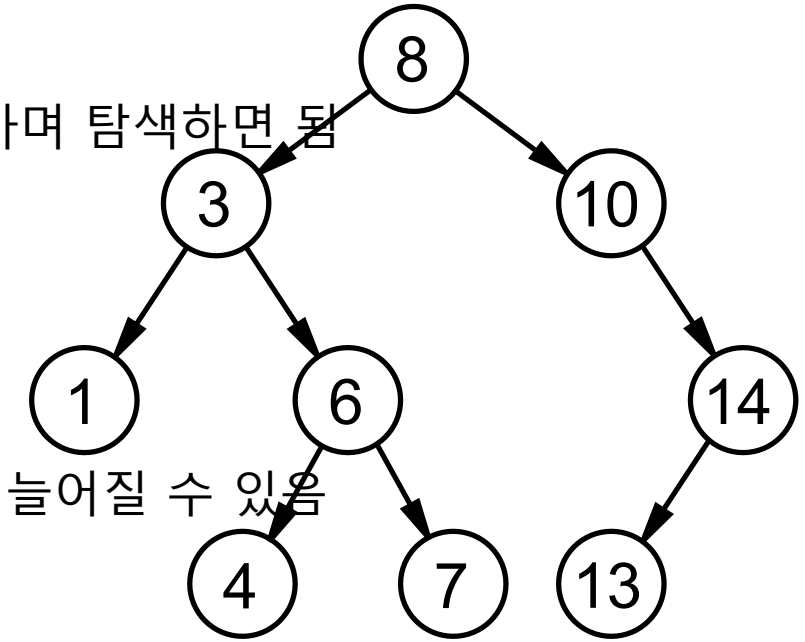
- hash 시간복잡도
  - 조회 : O(1)
  - 삽입 : amortized(O(1))
  - 삭제 : amortized(O(1))
- 특징 및 단점
  - 역연산 불가능 : 해시 함수는 key를 고정 길이로 변환 + 복잡한 연산 -> 정보 손실 발생 및 리버싱의 어려움 발생
  - 충돌 발생 : 서로 다른 key가 동일한 해시 값으로 변환될 수 있음 -> 충돌이 많을 수록 성능 저하
  - 순서 or 범위 검색 불가능 : 해시 값은 정렬되어 있지 않음

구현체 - BST

- 이진 탐색 트리(Binary Search Tree)란?
  - 각 노드가 하나의 key를 가지고, **"왼쪽 서브 트리의 모든 key < 현재 key < 오른쪽 서브 트리의 모든 key"**를 만족하는 이진 트리

    ※ 각 노드가 최대 2개의 자식 노드를 갖는 트리를 이진 트리라고 함
  - 따라서 key를 조회할 때 key 값에 따라 왼쪽 혹은 오른쪽으로 내려가며 탐색하면 됨

    ※ 가령, 오른쪽 트리에서 4를 찾는다면 8 -> 3 -> 6 -> 4로 이동하여 찾을 수 있음



- 이진 탐색 트리의 문제점
  - Key의 삽입 순서에 따라서 트리의 모양이 달라짐 -> 한쪽으로 길게 늘어질 수 있음

    ※ 이러한 모양의 이진 탐색 트리는 검색이 비효율적
  - 이러한 트리는 높이가 높아 탐색, 삽입, 삭제가 비효율적

구현체 - BST

- BST(Balanced BST)란?
  - 높이가 **O(log N)**이 되도록 유지되는 이진 탐색 트리
  - AVL 트리, Red-Black 트리 등이 존재
- BST의 시간복잡도
  - 이진 트리에서의 삽입, 삭제, 검색의 최악의 경우는 O(트리의 높이)
  - 따라서 삽입 삭제 검색 모두 O(log N)의 시간 복잡도를 가짐

    ※ 높이를 log N 수준으로 유지하는 회전 알고리즘 또한 log N의 시간 복잡도를 가짐

구현체 - BST

- set, map 의 구현체
  - STL의 set, map는 BST로 구현되어 있음
  - 따라서 set, map의 insert, find, erase의 시간 복잡도는 O(log N)